

it run quicker, overall. Hence, more and more software developers are turning to writing multi-threaded code in order to improve their application performance on multi-core processors.

One of the earliest and best articles to call attention to this paradigm shift was "The Free Lunch Is Over: A Fundamental Turn Toward Concurrency in Software" by the well known software guru, Herb Sutter. Interested readers can find this at www.gotw.ca/publications/concurrency-ddj.htm. It is a must read for every software developer who plans to write multi-threaded code.

Why is writing multi-threaded code difficult?

Writing robust multi-threaded code that gives a good performance is totally a different ball game. It is not just learning a new API or new language. It requires a major shift in the mindset of developers, since for every line of code in an application, developers need to answer the following questions:

- Can more than one thread execute this line of code simultaneously?
- What is the data getting read/written in this line of code?
- Can some other thread read/modify this data simultaneously?
- How do I ensure that my application is making forward progress?
- Can this line of code containing a call to `lock_acquire` lead to deadlocking my application?

Most programmers are conditioned to think in sequential mode. It is much easier to envisage an event 'A' as happening sequentially after an event 'B' instead of having to visualise whether parts of event 'A' can be interleaved with parts of event 'B'. However, since thread scheduling is non-deterministic, in order to facilitate robustness, multi-threaded code development requires that access to shared resources including data, should be protected by means of synchronisation. Threads also need to communicate with each other and exchange data, as part of the application logic.

The requirement that threads need to use proper synchronisation in order to protect shared accesses leads to two of the most dreaded problems in multi-threaded software development, namely, (a) race condition and (b), deadlock.

Data race conditions

Consider the following piece of code:

```
static class TicketCounter {
    static int curr_ticket_counter = 0;
    static int GetNextTicketId() {
        return curr_ticket_counter++;
    }
}
```

While the above snippet of code would be perfectly correct in providing a unique ticket ID to each caller in a single-threaded application, do you see what can happen if it is invoked in a multi-threaded application where more than one thread can call the `GetNextTicketId` function concurrently? It is possible that two threads are given the same ticket ID in this case. How can this happen? The single line of code `'return curr_ticket_counter++'` actually consists of the following three actions:

- Read the current value from the shared `curr_ticket_counter` variable into a register.
- Add 1 to that register value.
- Write the register value back to the shared variable `curr_ticket_counter`.

Two threads executing this same function concurrently can both read the same value from `curr_ticket_counter` (say, 100), increment their local register to 101, and publish the same resulting value. Hence both threads get the value of 100 from the function. Why did this happen? This inconsistency occurred because the simple statement `'curr_ticket_counter++'` appears atomic to the developer writing the function `'GetNextTicketId'` whereas it is actually not atomic. Hence, there is a race on the shared variable `'curr_ticket_counter'` and this code has a race condition.

A data race occurs in a multi-threaded program when two threads access the same memory location without any intervening synchronisation operations, and at least one of the accesses is a write.

Now, how can we modify the above code to avoid the data race on the variable `'curr_ticket_counter'`? The obvious answer is synchronisation, namely, using a lock to protect the access to the shared variable:

```
static int GetNextTicketId()
{
    int my_ticket;
    pthread_mutex_lock(&i);
    my_ticket = curr_ticket_counter++;
    pthread_mutex_unlock(&i);
    return my_ticket;
}
```

Can we assume that if we lock before accessing the shared variable and unlock after accessing the shared variable meticulously, our multi-threaded code will always work correctly? The answer is not as simple as that. You can use lock/unlock function around every shared variable access and still end up with an inconsistent multi-threaded code. Consider the following code snippet:

```
void Withdraw (int account, int money)
{
    pthread_mutex_lock(&account_lock /* for that account */);
    //get the curr_balance for this account and increment it
```